

Creative Coding at MSU

ARFD106 / VCDS225 / ARGS260

CLIENT

Montclair State University

YEAR

2019–2025

From 2019 to 2025 I developed and taught two creative coding courses at Montclair State University: Digital Literacy (ARFD106), a foundation-level course I first taught in 2019, and Creative Coding for Graphic Designers and for Artists (VCDS225/ARGS260), which I developed and taught from 2022. Both courses were built for students in the College of the Arts who had no programming background and, in many cases, no particular desire to acquire one. They were designed not as introductions to computer science but as expansions of students' existing creative practice: coding as another material to think with.

The Design Premise

The central problem in teaching creative coding to art and design students is that the frame through which most coding instruction is delivered is wrong for the audience. A computer science framing, even a creative one, asks students to become something they are not. Art and design students have already developed sophisticated ways of making, thinking about form, working iteratively, and tolerating ambiguity. The question is not whether they can code. It is whether the code can do something for them.

The courses at MSU were built on that inversion. Rather than starting with concepts and looking for creative applications, they started with what students already cared about making: their own photographic practice, their graphic work, their interest in interactivity or data or generative form. Code was introduced as the thing that could extend those practices, not replace them. The concepts followed from the making rather than preceding it.

This is the same design logic that shaped the K-12 Vidcode curriculum. Arrays are not introduced as a data structure; they are introduced as a list of things that matter to the student. The difference in a college art context is that the students arrive with a more fully developed sense of what they are trying to make, and with a higher tolerance for ambiguity and self-direction. The scaffolding can be lighter; the open-ended work can start earlier; the critical conversation about what the code is doing aesthetically and conceptually can be sustained at a higher level.

Creative Coding for Graphic Designers and for Artists

The Creative Coding courses (VCDS225 for graphic design students, ARGS260 for studio arts students) were more advanced and more explicitly studio-based. They were designed as creative coding practice courses: students coded, but the work was assessed and critiqued as creative work, not evaluated as correct or incorrect solutions to technical problems.

The courses were structured around projects that required students to develop a concept, choose appropriate tools, build something, and present the work in a critique context. The technical content was introduced in relation to what the projects demanded: event listeners and interactivity when students were building things that responded to input; generative methods and randomness when students were exploring pattern and variation; data structures when students were working with collections of images or texts. The sequence was driven by practice rather than by a predetermined curriculum arc.

Assessment reflected the studio model: students were evaluated on the ambition and coherence of the concept, the craft of the execution, and the quality of their reflective practice, not primarily on whether the code worked correctly. Code that ran but did nothing interesting was worth less than code that was broken in an interesting way and accompanied by a clear account of what the student was trying to do.

Reflection

The tension that recurred throughout both courses was between the technical knowledge students needed and the pace at which the creative work demanded moving. Students who were comfortable with ambiguity and open-ended briefs sometimes found the specificity of code frustrating: it does exactly what you tell it, and what you tell it and what you mean are often not the same thing. Students who were comfortable with technical problem-solving sometimes found the open-ended creative briefs disorienting: there is no correct answer, and the absence of a correct answer is the point.

Managing that tension was the core pedagogical challenge. The most effective approach was a tight iteration cycle: small experiments, frequent sharing, critique focused on what the work was trying to do rather than whether it succeeded. The studio critique model, applied to coding work, turned out to be well suited to the problem. It normalized incompleteness, valued intention, and kept the conversation about meaning rather than about whether the code ran.
